

Final Project Overview

This project focuses on predicting whether a hotel booking will be canceled using machine learning classification models. The main goal was to compare multiple models, evaluate their performance using different metrics, and determine which approach works best for this type of problem. The project also includes building ensemble models to improve prediction accuracy and stability. Overall, this project demonstrates the full machine learning workflow, including data preprocessing, model training, evaluation, and analysis.

Dataset Source

- Original dataset: Hotel Booking Demand dataset from Kaggle
- Dataset used for this project: hotel_bookings.csv

What I Accomplished

- Prepared and cleaned the dataset for analysis
- Handled categorical variables using encoding techniques
- Split the data into training (70%), validation (15%), and test (15%) sets
- Trained six classification models:
 - Logistic Regression
 - Decision Tree
 - Random Forest
 - XGBoost
 - K-Nearest Neighbors (KNN)
 - Support Vector Classifier (SVC)
- Evaluated all models using multiple performance metrics:
 - Accuracy
 - Precision
 - Recall
 - F1-score
 - ROC-AUC

- Created visualizations to analyze model performance:
 - ROC curves
 - Confusion matrices
 - Bar charts
- Selected the top three models (XGBoost, Random Forest, and SVC) based on performance
- Built ensemble models using:
 - Voting Classifier
 - Bayesian (weighted average) approach
- Compared ensemble models with individual models to evaluate improvements
- Analyzed results to identify the best-performing model and understand strengths and weaknesses

Discussion and Analysis

The goal of this project was to predict whether a hotel booking would be canceled using classification models. To do this, I first prepared the data by cleaning it, handling categorical variables using encoding, and splitting the dataset into training (70%), validation (15%), and test (15%) sets. This step was important because it allowed me to train the models properly and evaluate them on unseen data.

I trained six models including Logistic Regression, Decision Tree, Random Forest, XGBoost, K-Nearest Neighbors (KNN), and Support Vector Classifier (SVC). Each model learns patterns in a different way. For example, Logistic Regression assumes a linear relationship, while tree-based models like Decision Tree and Random Forest can capture more complex patterns. XGBoost builds models sequentially and improves performance by correcting previous errors. SVC, especially with the RBF kernel, can model nonlinear decision boundaries. After training all models, I evaluated them using multiple metrics such as accuracy, precision, recall, F1-score, and ROC-AUC. Using multiple metrics helped me better understand model performance instead of relying on just one number.

XGBoost performed the best overall. It had the highest F1-score and ROC-AUC, which means it was both accurate and effective at distinguishing between canceled and non-canceled bookings. This shows that XGBoost was able to capture complex relationships in the data and reduce errors effectively. Random Forest also performed well and had very high precision, but its recall was much lower. This means it was very careful when predicting cancellations but missed many actual canceled bookings. SVC ranked next and showed a good balance between precision and recall, with a strong ROC-AUC, making it a solid model for this problem. Decision Tree had a

more balanced performance but was weaker overall compared to SVC. Logistic Regression and KNN had lower performance.

Then, I created ensemble models using the top three models: XGBoost, Random Forest, and SVC. I used both a Voting Classifier and a Bayesian (weighted average) approach. The idea behind ensemble methods is that combining multiple models can produce more stable and reliable predictions. The results showed that the ensemble models performed similarly or slightly better than the best individual model, especially in terms of consistency between validation and test sets. This indicates that the ensemble approach helped reduce overfitting and improved generalization.

Conclusion

This project helped me understand that building a machine learning model involves much more than just training one algorithm. It requires careful data preprocessing, testing multiple models, and evaluating results using different metrics. This project also helped me understand that there is no single best model in all situations. It is important to compare different models, evaluate them using multiple metrics, and sometimes combine them to achieve better results.